让技能进入下一次 rollout

基于 VERL 与 Harvey 的技能演化 harness

设计与实现说明 2026 年 10 月 29 日

这份作业关注一个具体问题：模型从一次任务中写出的经验，怎样进入后续任务，又怎样把后续收益归给写经验的那次行为。我在 VERL 的 AgentLoop 上加入技能快照、Writer 候选和组间发布协议，用 Harvey 法律文档任务检查这条链路。本文保留两组有代表性的结果：五轮实验中第一轮的正向发布，以及最后一版多次采样实验的负结果。

当前交付包括代码改动、启动 README、设计说明和完整轨迹。正文按原生 vLLM/SGLang rollout 的接法说明 Ray worker、工具状态机和训练张量如何衔接；实验验证生成、加载和发布协议。是否完成 GPU 训练不作为本文的主要讨论对象，已测范围在实验节统一说明。

1 我的出发点：写出经验和用好经验，需要分开评价

我最初关心的不是怎样把更多文字塞进 prompt，而是怎样判断一条经验值得留下。来源案件做对了，只能说明这次求解成功；从中提炼出的步骤可能过于具体，也可能只是模型本来就会的常识。反过来，来源案件做错了，失败中仍可能提炼出有用的检查方法。因此，我把 Solver 和 Skill Writer 的信用分开：Solver 对当前案件的交付结果负责，Writer 对技能在其他案件中的边际作用负责。

这个区分决定了后面的工程结构。Writer 不应靠“生成了一条 skill”得分，也不能直接继承来源案件的分数。它先提出一个维护动作，再让其他案件分别读取旧库与试验库；两者的结果差，才是这次维护动作的评价信号。检索命中只是中间事件。技能被选中后仍可能增加冗余、占用上下文，甚至挤掉更有用的旧技能。

第二个判断是，技能库版本属于 rollout 条件。GRPO 同组样本需要在可比的环境中采样；如果并发 worker 随时改写共享库，后完成的样本会看到先完成样本写入的内容，组内差异便混入了执行顺序。我的处理是每组固定模型、技能快照和检索策略，worker 只读；候选先隔离，等这一组结束后再由协调者决定发布。下一组才读取新版本。

第三个判断是，要分别观察“技能内容更好”和“模型更会使用技能”。设计中保留冻结执行模型与当前模型的四格评价：在两个模型下分别比较旧库与新库。冻结模型提供较稳定的内容评价，当前模型反映实际组合效果。两者仍有交互，不能仅靠相减就声称完全分离了两种能力。本次只测了固定执行模型下的库差，四格是后续训练要补的实验。

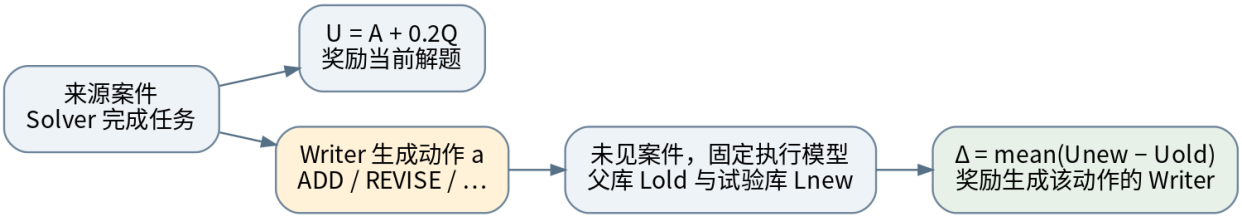


图 1 奖励分给当前解题与后续技能收益

Voyager 展示了通过外部技能库积累和复用行为的路径；SkillRL 将经验蒸馏为分层 SkillBank，并与策略训练结合；Evolving-RL 进一步讨论经验提取和使用的联合优化。我借鉴这些方向，但这次实现主要解决它们落到 VERL 后的接口问题：快照在哪里固定，候选由谁发布，延迟分数怎样回到 Writer 的生成 token。相关工作见文末参考资料，不直接比较不同 benchmark 的成绩。

1.1 从已有做法走到这里

早期把完整轨迹或反思文字存入记忆的做法，便于找回案例，但复用的内容可能混有案件事实、偶然动作与错误解释。我希望留下的是可执行的检查程序，因此先区分工作记忆和持久技能，再把技能的适用边界写进结构。这个选择也带来代价：抽象时可能丢掉任务成立所需的条件，必须用其他案件验证。

已有方向	借鉴的部分	这次进一步要解决的问题
Voyager [1]	技能积累、检索和执行反馈；以可执行程序保存行为	外部库能复用经验，但在 VERL 中还需要定义哪次写技能的动作承担 RL 信用，以及并发 worker 何时看到新版
SkillRL [2]	经验蒸馏为分层 SkillBank，检索通用与任务特定启发式，并与策略共同演化	分层有助于组织，仍需检查某次库更新的收益来自技能内容、使用模型，还是检索竞争；本文固定版本并保留四格评价设计
Evolving-RL [3]	从评价中分别取得提取者与求解者的学习信号，联合改进经验提取与使用	信用拆分需要落到实际样本身份、模型 token、reward 账本及 Trainer 边界，不能只停留在评价公式

我的 insight 放在这些做法之间的接口上：把“写经验”和“使用经验”变成两个可评价的策略行为，用来源案件之外的边际收益评价 Writer；再把技能快照当成训练环境版本，确保延迟评价仍能归回产生它的原样本。四格不是声称完全解耦内容与能力，而是让两种变化至少可分别观察。本文不声称重新发明技能库或提取者/使用者双角色，贡献集中在这套归因与并行 rollout 协议的设计和接线。

2 原来的 rollout 为什么不能直接完成这件事

需要先把“无状态”说清楚。本 checkout 的 VERL 已经支持多轮工具调用。ToolAgentLoop 会保留当前样本的消息、工具观察和生成 token，并在生成与工具执行之间切换。这里缺少的是跨样本的技能生命周期：一条轨迹结束后，没有默认协议把其中的经验变成候选、评价候选、发布版本，再让下一组 worker 读取。

原生数据流是 Trainer 把任务组成 DataProto，AgentLoopManager 分发给 Ray AgentLoopWorker，worker 并发运行样本，输出 token、response_mask 和 reward，随后进入 advantage 与 actor loss。这个流围绕当前样本设计。若只在求解 prompt 末尾加一句“总结经验”，得到的仍是这条样本的输出；它既没有稳定的技能身份，也没有后续读取入口，更没有跨案件的信用归因。

直接让 worker 写一个共享 JSON 文件也不够。多个 Writer 可能同时修改同一条技能；验证阶段可能把候选误写进活跃库；一个基于旧快照生成的迟到动作可能覆盖新版；同一组四次采样也可能读到不同库。这些都是训练环境变化，不能交给“最后一个写文件的人”决定。

我保留 VERL 已有的 Ray 调度、模型服务客户端、ToolAgentLoop 状态机、DataProto、reward 和 PPO/GRPO 接口，在样本入口增加快照读取，在组外增加候选协调。原生训练路径仍由 AgentLoopManager 创建 Ray workers，再实例化注册的 HarveySkillAgentLoop。技能内容不放进 Ray actor 的隐式可变状态，而通过配置显式指定路径和 expected_snapshot_id。

2.1 接到 vLLM 的位置

入口仍是 verl.trainer.main_ppo。启动脚本默认设置 rollout.name=vllm、多轮工具调用和 harvey_skill_agent；AgentLoopManager 使用原生 Ray actor 创建 AgentLoopWorker，并把模型服务客户端传入自定义 loop。HarveySkillAgentLoop 继承 ToolAgentLoop，在自己的 run 中完成任务环境和技能注入，再调用 super().run。父类的生成状态通过 server_manager.generate 请求 VERL 管理的 vLLM rollout replica，返回 TokenOutput；不在 Harvey 适配层重新创建 vLLM 引擎，也不改写 vLLM 的采样或 KV cache 实现。

一次任务的顺序是：任务 ID 和父快照进入 worker；先检索、组装技能上下文，再对完整消息做 chat template 和 tokenization；生成阶段交给原生模型服务；工具调用由本轨迹的 Harvey executor 执行，结果以观察消息追加；父类继续生成直到 finish 或预算结束。模型生成 token 的 response_mask 为 1，工具观察和 padding 为 0。技能文本在输入侧，不是 Writer 此次生成的动作，也不应成为作者梯度位置。

这个接点保留了 Ray 资源放置、AgentLoopWorker 内的协程并发、rollout replica 生命周期，以及 Trainer 的 actor 权重同步和 GRPO 更新基础设施。actor checkpoint 在同步训练窗口内固定，窗口结束才更新。快照路径、expected_snapshot_id 和 Router/model 标识也在同一窗口固定，避免模型更新与技能更新错配。

2.2 把跨 rollout 状态接回训练 step

一个训练 step 内的工具 turn 可以很多次，但读取的技能快照不随 turn 变化。Solver 完成来源任务后，Writer 把轨迹抽象成一个维护动作；动作写入 staging，当前 step 的其他样本仍读取父库。候选的验证可以异步执行，但 actor 的该组 reward 未齐时不能提前结算 advantage，也不能把未完成候选当零分。协调者收齐评分后做发布判断；下一训练窗口使用新 group config 和新 expected_snapshot_id。

当前快照在 loop 构造时加载，发布器位于组外。原型脚本通过组间重新生成配置并重建 worker 来加载下一版；尚未做 Trainer 内的自动热切换。不能只修改 worker 正在读取的 JSON 文件，期待已加载对象自动刷新。后续若把发布屏障直接接进 Trainer，应在 generate_sequences 返回、该组 reward 完成和 actor 更新边界之间明确记录模型版本与下一快照，重建或显式刷新所有 worker 后再发下一组。

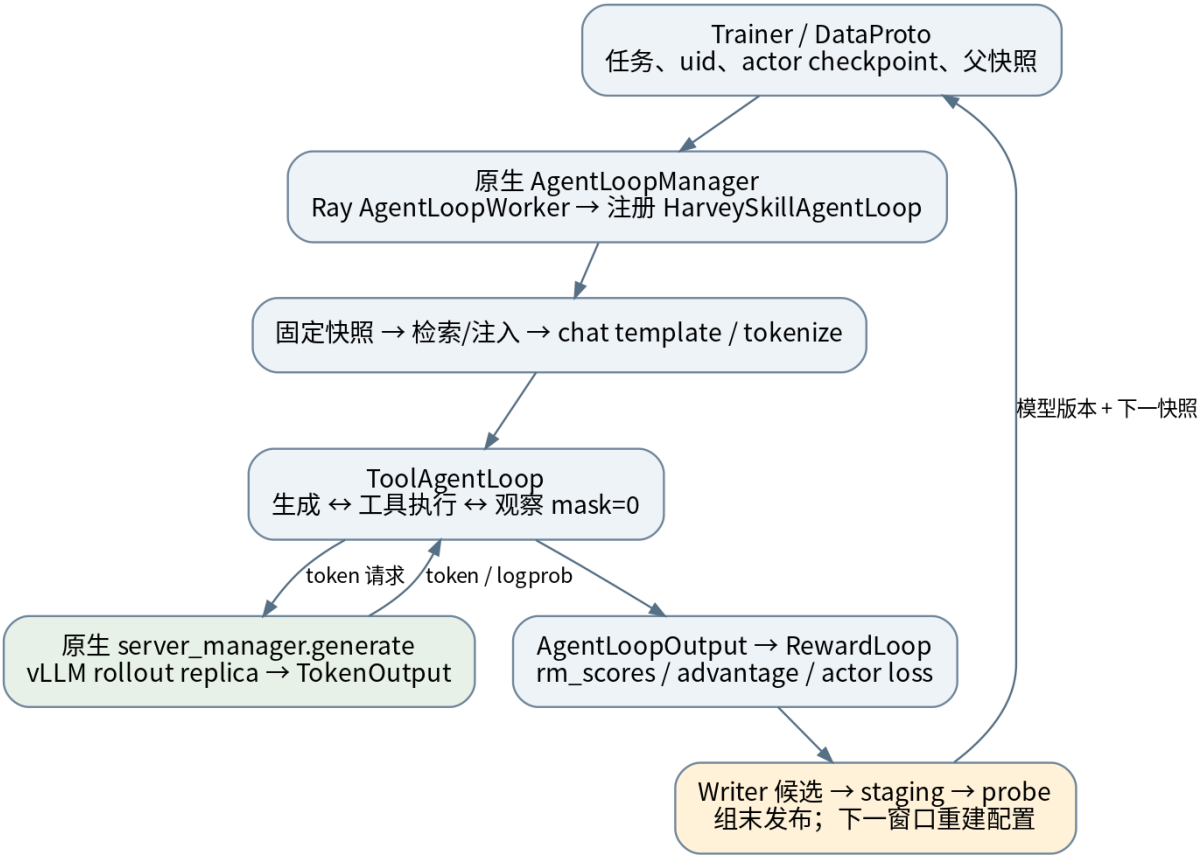


图 2 原生 vLLM 多轮 rollout 与组间技能发布

3 reward：当前案件得分，后续案件归因

Harvey 的交付物包括抽取表、风险分析和起草文件，适合检查步骤能否跨案件复用。任务按“ 操作模式 × 法律领域 ”组织，例如 extract × real-estate、draft × employment-labor。当前来源与验证均从 Harvey 任务目录划分，尚未接入之前的造题 pipeline；目录互斥是已执行的隔离，源文档和模板家族的严格去重仍需补齐。

任务效用定义为：

```
A = 1[所有 criterion 均通过]
Q = 通过 criterion 数 / criterion 总数
U = A + 0.2 × Q
R_solver = U(当前案件)
```

all-pass 保留任务完成的主目标，Q 让未全通过的样本仍有较密的质量反馈。这里必须从 n_passed/n_criteria 计算 Q；Harvey 的 score/max_score 在这套评分中对应 all-pass，不能拿来代替通过比例。工具调用、技能加载和候选条数都不单独加分。

Writer 的评价在来源案件之外计算。对维护动作 a，从同一个父库分出 L_new(a)，其余模型、任务、工具预算和检索算法固定：

```
Δ_ref = mean_x[U(x; M_ref, L_new) - U(x; M_ref, L_old)]
Δ_cur = mean_x[U(x; M_cur, L_new) - U(x; M_cur, L_old)]
R_writer = α × Δ_ref + (1- α ) × Δ_cur - C_skill - N_transfer
```

M_ref 是冻结的执行模型，与 PPO reference policy 含义不同。C_skill 表示技能带来的额外成本，N_transfer 表示负迁移。writer_policy.py 保留四格计算接口；本次实际标量只有 Δ_ref，成本项尚未加入奖励，负迁移通过发布硬门槛处理。发布要求原始平均收益为正且每道验证题不退步。GRPO 的组内中心化 advantage 只能用于优化，不能代替这项发布判断：即使所有候选都变差，仍可能有候选获得正的相对 advantage。

3.1 reward 怎样回到 Writer 的 token

Writer 是独立的 SingleTurnAgentLoop 样本，输出一个 JSON 维护动作。来源任务、父快照和 uid 定义一个候选比较组，rollout.n 产生多个候选；ADD、REVISE、MERGE 或 RETIRE 都可以在同组比较，只要它们基于相同来源与父库，并使用同一批 probe、模型与预算。不同 step 的任务难度和库版本不同，不把它们的 reward 拼成一个 GRPO 组。

评价端为每个 candidate_id 建立独立 trial，在未见案件上执行旧库与新库 rollout，再由 Harvey criterion judge 评分。每道题先对重复样本求均值，再对题求均值；不能因为某候选生成了更多条轨迹，就让这道题获得更大权重。缓存旧库基线必须绑定任务、actor checkpoint、父快照、检索版本、采样与工具预算及 judge 版本，不能直接拿上一 step 的旧分数复用。

RewardLoop 的接口返回 score 与分解字段：candidate_id、parent/trial、各题 U_old/U_new、Δ_ref/Δ_cur、成本和负迁移项。框架将标量组装为终局 rm_scores，绑定的是原始 Writer 的 uid/session，而不是 probe 的求解 token。probe 中的 Solver 也有自己的 U，但这些观察不能冒充 Writer 的策略动作。reward trace 同时记录 response token、response_mask、skill_author_mask、policy_loss_mask 与非零 reward 位置，用来检查延迟分数是否回到了原样本。

```
Writer 响应：[ ADD / REVISE / JSON 内容 ... 结束 ]
response_mask：[ 1 1 1 1 ... 1 ]
author_mask：[ 1 1 1 1 ... 1 ]
rm_scores：[ 0 0 0 0 ... R_writer ]
输入技能 / 工具观察：不属于此次 Writer 作者 token
```

当前 SingleTurn Writer 的 author mask 覆盖整段生成响应。TransferQueue 校验 mask 长度、作者 token 范围和 reward 位置；PPO loss 使用 policy_loss_mask。advantage 仍由 VERL 原生路径计算。以后若让同一长轨迹内交错求解和写技能，必须进一步标注多个作者片段并处理各自 credit，不能沿用“整段响应都是 Writer”的假设。

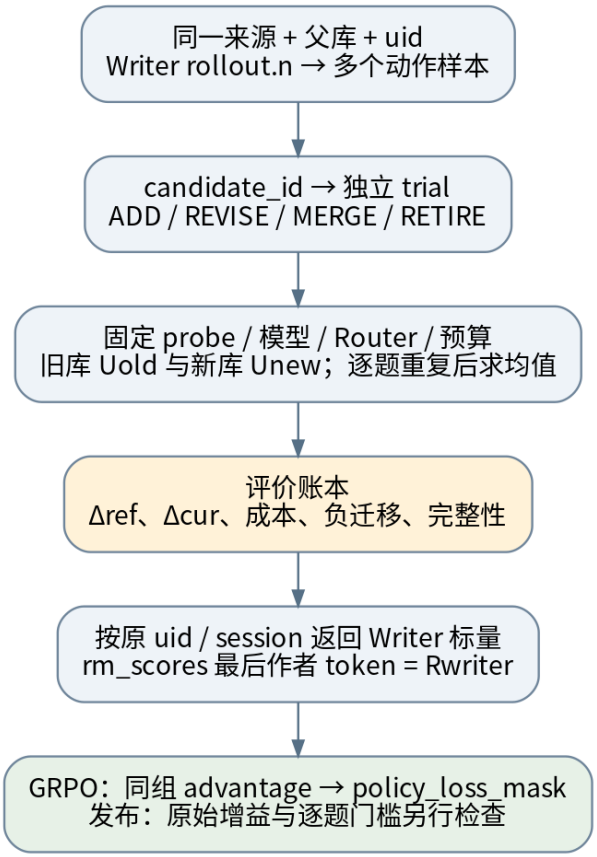


图 3 候选 probe、轨迹级信用与原生训练张量的连接

3.2 不同技能动作与异常结果怎样计分

同一个 step 的 skill 改动幅度可以不同，评价目标仍是固定父库上的边际效果，而不是“改了多少字”。重复动作按内容哈希去重，共享相同评价结果并保留原采样的 uid；不能把重复候选删掉后悄悄改变原 GRPO 组。NO_CHANGE 提供零变化基线，无效 JSON 或非法目标可给确定的结构负分。工具调用多少、ADD 还是 REVISE 本身不加奖励。

成本项应使用相对父库增加的注入 token、求解 token 或延迟，并在开发集固定系数。一次性离线验证的费用与部署时的持续使用费用应分开记录。现在实测标量仍只有平均质量差，成本项为零；逐题不退步属于独立发布门槛，尚未作为可微训练中的惩罚项实测。

超时需要区分原因：模型在规定工具/turn 预算内未完成属于任务结果，按统一规则评价；机器故障、服务断连、judge 缺失则是评价未完成，应按原 candidate_id 和固定条件重试，不能当模型负分或静默补零。评价任务齐全后才结算同组 advantage。probe 应使用受限的独立评价资源，避免 Writer reward 等待 probe 时占住全部 rollout 资源而形成资源饥饿。原生 probe 调度到专用 Ray/vLLM worker 池是下一步接线；现有 custom reward 已实现候选隔离、配对计算与返回分解值，尚未完成该原生评价池适配。

验证题用于决定发布，就属于确认集，不能再称完全未触碰的最终测试集。早期五轮还用验证题的检索可达性预筛过候选；最后一版已经改为只看来源任务。两版结果都只能作小规模协议验证，不支持统计显著或普遍泛化的结论。

4 skill 怎样存、怎样读、怎样更新

SkillBank 分为 General、Pattern 和 Legal Domain。General 保存证据核验、交付检查等跨任务步骤；Pattern 保存抽取、比对、起草等操作程序；Domain 保存特定领域的检查项。组合型技能需要更窄的适用条件。持久内容应写步骤、前置条件和不适用边界，案件当事人、金额、答案及隐藏 rubric 留在案件工作记忆中。Writer prompt 明确了这项约束，目前仍需人工审阅，尚无可靠的自动语义泄漏检测器。

SkillSnapshot 在加载时计算内容哈希和快照 ID，后续文件变化不会改变已经加载的对象。每条轨迹记录实际使用的技能 ID、内容哈希和快照。最后一版检索先按操作、领域和 exclusions 硬过滤，用本地词/字符特征哈希向量的余弦分数与词法分数混合召回最多 8 条，再由独立版本的规则 Router 选最多 3 条，受估算 1500 token 预算约束。这是可复现的初版，不是训练过的语义 embedding 或 LLM Router。

同题四次采样共享按任务、query 哈希、模型标识、快照、Router 与索引版本生成的选择键。Solver 的后续采样仍独立。这样，重复采样主要观察解题波动，避免同时混入检索变化。

4.1 初始 skill 从哪里来

初始库分两次构造。先将本项目的可演化 skill 快照置空，运行五个法律领域各两条来源任务，共 10 条 rollout。Harvey 自带的工具说明、文档处理手册和任务 prompt 仍照常加载；这里的“空库”只指新增的可演化 SkillBank。这样先收集模型原有的操作过程，再提炼跨案件步骤，避免把人工编好的领域答案当成学习结果。

bootstrap_skill_bank.py 读取每条来源的快照 ID 和完整轨迹，确认都基于空快照、任务完成且覆盖五个领域。提炼输入不是把整份案件材料原样拼接，而是从 assistant 记录中取动作摘要、turn 和工具名：每段文本最多 300 字符，一条轨迹最多保留前 12 与后 12 个助手动作。初版蒸馏输入没有直接加入 rubric 文字，也没有读取验证题。这个压缩便于控制上下文，但可能丢失轨迹中段的重要修复过程，是 seed 质量的一项局限。

分层结构由方案预先设定，模型在这个结构中写内容。第一版要求正好 8 条：1 条 General、extract 和 identify 两条 Pattern，以及房地产、保险、公司并购、劳动雇佣、诉讼争议解决五条 Domain。蒸馏温度固定为 0，步骤要求可执行、带触发和排除条件，禁止复制案件事实、金额、日期、答案和 rubric 表述。随后用 Skill.from_dict 检查结构、ID 唯一性、层级标签和领域覆盖；失败记录原因，最多重试三次。代码检查的是结构，语义上是否泄漏或过度概括仍需人工审阅。

种子阶段	提炼证据与预设结构	保存结果
第一版 seed	10 条空库来源；1 General + 2 Pattern + 5 Domain	8 条，harvey-seed-v0001-84e13fea099a
扩充 seed	新增房地产/劳动雇佣共 20 条空库来源；补 analyze、compare、draft 各一条 Pattern	保留原 8 条并加入 3 条，共 11 条，harvey-seed30-v0001-07465a80603a

第二次由 extend_bootstrap_extra20.py 执行。它校验 20 条来源的空快照和评分，并要求每个新增操作至少有四条来源；提炼使用目标操作的动作摘要、criterion 通过比例，以及每条最多四个失败项的标题和简短原因。这里使用的是来源题的 rubric 反馈，不是后续验证题。提示要求把这些信号改写成通用检查步骤，禁止照搬 rubric、judge 推理或案件事实；结构验证后再人工检查内容。新增技能是 analyze-redline-risk-memo、compare-baseline-deviation-matrix 和 draft-source-grounded-instrument。

两次快照都保存 source_runs 和 provisional_seed 状态，按规范化内容计算哈希并以新文件写入。provisional 表示可作为初始指导，不表示每条技能都已证明有增益；后续 Writer 的独立案件评价负责逐步检验和修订。首轮正向实验使用 8 条 seed，最后一版使用 11 条 seed。初始层级和数量是人工设定的归纳偏置，具体步骤来自模型提炼，不能把这次 bootstrap 描述成自动发现了最优技能分类。

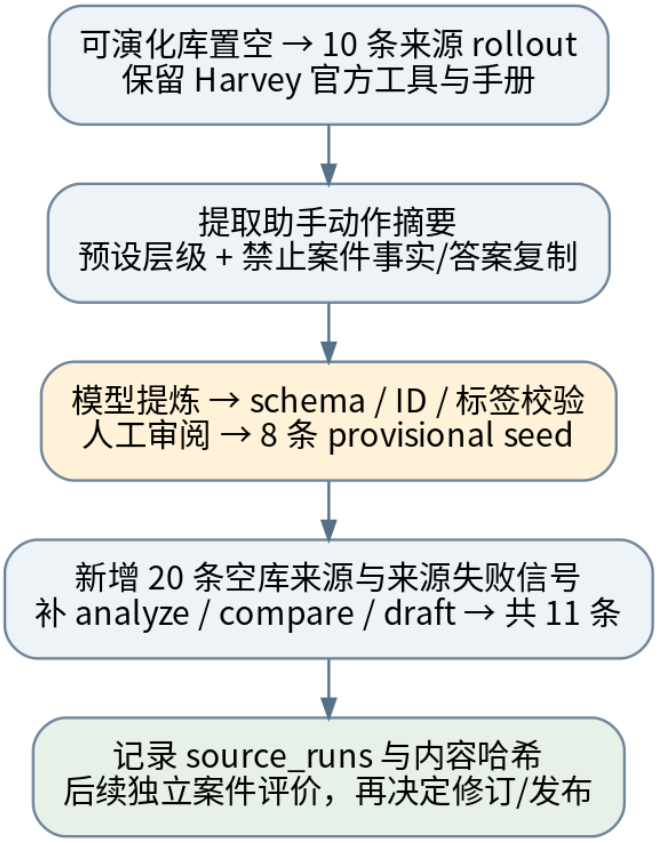


图 4 来源轨迹到暂定 seed，再到后续收益验证

4.2 先区分读取冲突和写入冲突

读取时，不同任务选择不同技能是正常情况；同题同组重复采样才要求选择一致。Router 按明确的 incompatible_skill_ids 排除不兼容组合，操作、领域和 exclusions 优先于相似度。未知的语义矛盾仍可能漏检，因此相似度高不能作为兼容证明。当前只实现显式关系和规则检查，隐含矛盾需要后续审阅或独立检测。

写入时，Writer 只产生 ADD、REVISE、MERGE、RETIRE、NO_CHANGE 候选，不直接写活跃库。候选带 parent_snapshot_id、目标 ID、动作内容哈希和来源轨迹。candidate_barrier.py 校验父版和内容哈希、去重，记录同目标、同输出 ID、显式不兼容和高词重合边。REVISE/RETIRE/MERGE 目标相交属于同一个写冲突，不能用写入时间决定胜者。

冲突情形	处理规则	当前边界
两个候选修订同一技能或生成同一 ID	在同一父库上保留两个 trial，禁止相互覆盖	已记录冲突图；当前实验只评价一个分支
完全重复的动作	按动作哈希复用评价，来源与原始样本身份保留	staging 去重已实现；训练样本重映射需随原生评价池补齐
基于旧父版的迟到动作	拒绝直接发布；需要重基并重新评价	父快照校验已实现
不同 ID 但步骤相反	显式不兼容关系先阻止共注入；不确定的候选隔离审查	隐含自然语言矛盾尚不能可靠识别
各自有效、组合后争夺 top-N	必须评价组合快照，不能相加单候选收益	当前一次仅发布一个动作，未实现组合发布

发布由单一协调者在窗口屏障执行。它核对原始逐题增益和 trial 父版，只发布一条合格分支；失败就保留父库。文件发布复制到独立 inode、校验、落盘，再建立只读发布文件。当前依赖单协调者部署约束，没有

分布式“当前版本指针”的 CAS；若扩到多个 Trainer，应由一个 Ray publisher actor 或事务存储串行执行 compare-and-swap，而不能各进程各自判断父版后发布。

5 具体改动：放在哪一层，为什么这样放

VERL 核心只改了四个文件，当前 diff 为 40 行新增、2 行替换。Harvey 适配和技能协议放在 examples/skill_evolving/，避免让通用训练代码依赖法律任务。完整补丁随交付包提供。

文件 / 入口	具体变化	放在这一层的原因
verl/experimental/agent_loop/agent_loop.py	调用自定义 loop 前写入 extra_info.phase	worker 已知道 train/validation；Writer 据此禁止验证期 stage
verl/trainer/ppo/v1/agent_loop_tq.py	检查 author mask 长度及范围，传递 policy_loss_mask，导出 reward trace	在轨迹转为训练张量时检查 token 对齐，避免错误一路进入 loss
verl/workers/utils/padding.py	去 padding 时优先使用 policy_loss_mask	保持打包后的 loss 范围与作者 token 一致
verl/workers/utils/losses.py	ppo_loss 选入并使用 policy_loss_mask；缺省回退 response_mask	复用现有 PPO loss，仅收窄 Writer 的优化位置

```
# AgentLoopWorker：传递阶段
+ extra_info["phase"] = "validation" if trajectory["validate"] else "train"
# TransferQueue：检查作者 mask 后随训练数据传递
+ field["policy_loss_mask"] = policy_mask
# padding / actor loss：普通任务仍可使用 response_mask
- data["loss_mask"] = data["response_mask"]
+ data["loss_mask"] = data.get("policy_loss_mask", data["response_mask"])
- response_mask = data["response_mask"].to(bool)
+ response_mask = data.get("policy_loss_mask", data["response_mask"]).to(bool)
```

上面只摘出关键行。TransferQueue 实际还检查 mask 不覆盖工具观察，并核对非零 reward 的位置。advantage 计算仍沿用原生 response_mask；当前单轮 Writer 的 author mask 覆盖整段生成 JSON，不是句法级的更细标注。原生更新前仍需检查整条 token 对齐路径。

示例层文件 / 函数	实现与接口
harvey_agent_loop.py / HarveySkillAgentLoop.run	继承 ToolAgentLoop；接收 extra_info.harvey_task_id，校验 expected_snapshot_id，创建私有沙箱，将 episode.messages 交回父类 run；返回 AgentLoopOutput 与检索来源
harvey_agent_loop.py / _call_tool	复用父类工具状态转换，把调用分发给本轨迹 ToolExecutor；轨迹内锁保护 cwd 和文件状态，semaphore 限制沙箱数量
harvey_episode.py / HarveyEpisode	统一 Harvey 任务、官方 prompt、工具 schema、技能注入和输出目录；工具观察上限统一为 65,536 字符
skill_store.py、skill_router.py	只读快照、稳定哈希、混合召回、兼容性与预算选择、同组选择缓存
writer_agent_loop.py / run	复用 SingleTurnAgentLoop 生成动作，训练期 stage，输出 candidate_id、父版、source_run_dirs 与 skill_author_mask
writer_reward_fn.py / compute_score	独立 probe 比较 old/new，返回 score 和 reward breakdown；原生 RewardLoop 再把分数组装进训练数据
candidate_barrier.py、commit_trial.py	候选隔离、冲突记录、父版核验与原子发布；发布权留在组间协调者
make_group_config.py	生成带快照 ID 的注册 loop 配置，供原生 AgentLoopManager/Ray worker 加载

示例层文件 / 函数	实现与接口
run_harvey_grpo.sh	默认选择 vLLM，多轮格式、注册 AgentLoop 与原生 Trainer 一起启动；可配置后端与 AgentLoop worker 数

这套分层把任务与技能协议接到原生 server_manager、Ray worker 和训练张量接口上。启动脚本现以 ROLLOUT_BACKEND=vllm 为默认，AGENT_NUM_WORKERS 控制外层协程 worker 数；不需要在 Harvey 适配层另写模型生成引擎。原生 probe 评价池与 Trainer 内发布屏障仍是下一步集成项。

6 实验：一次通过发布，一次被拒绝

两组验证固定执行模型与 Harvey judge，使用相同任务效用公式。这里主要看候选是否生成、后续是否加载，以及奖励和发布判断是否一致，不展开全部运行参数。原始快照、逐项评分和完整 JSONL 见交付索引。

6.1 五轮实验第一轮：领域技能修订获得正收益

这一轮从房地产来源任务中修订 dom-real-estate-review，增加 controlling-value reconciliation ledger：每个关键值记录权威来源、冲突值和处理依据；成品提交前再从交付物反查来源。候选在两个未见案件上比较旧库和试验库。

验证案件	old 通过数	new 通过数	old U	new U	差值
规划批准条件抽取	42/47	43/47	0.178723	0.182979	+0.004255
现有租约组合条款抽取	90/94	93/94	0.191489	0.197872	+0.006383
平均	—	—	0.185106	0.190426	+0.005319

两题均未 all-pass，提升来自 criterion 通过率。Writer 的固定执行模型奖励为 +0.005319，候选通过逐题非退化门槛并发布。第三轮的 zoning-compliance 训练轨迹加载了该领域技能的新哈希 cec5efdba6d0…，证明发布内容进入了后续 rollout。这条证据支持跨轮复用协议跑通；单次比较、且早期存在验证可达性预筛，不能据此认定稳定收益。

6.2 最后一版：新技能被加载，但整体收益为负

最后一版用一题四次来源轨迹提出动作，再在两题上 old/new 各运行四次，共 20 条轨迹。20 条均正常 finish，评分错误为零，反馈哈希和权限均通过核验；相关代码测试 18 个通过。

Writer 的 8 个输出中有 7 个合法候选，屏障记录 3 条同输出 ID 冲突。选出的 ADD 动作新增 draft-real-estate-instrument。它在 8/8 条 new 验证轨迹中进入 top-3，同时挤掉 dom-real-estate-review。

验证案件	old 四次均值	new 四次均值	差值
贷款协议起草	0.191667	0.190476	-0.001190
抵押与担保协议起草	0.191304	0.191848	+0.000543
总均值	0.191486	0.191162	-0.0003235

这一轮未发布，下一组仍使用父库。同题四次 Router 选择键与技能哈希一致，说明比较过程中检索没有组内漂移。这里最有价值的现象是 top-3 的替换：新技能步骤看起来合理，但其净作用还包括被挤掉旧技能的机会成本。奖励必须评价“ 模型 + Router + 整个库 ”的任务结果，不能只给单条技能的文字质量打分。

两组 old/new 都是独立采样，序号不代表共同随机种子。正负结果幅度都小，确认题也少。本文把它们作为机制验证和问题定位，不作显著性结论。第一组检查共享任务协议和跨轮发布，最后一组进一步检查 VERL ToolAgentLoop 状态机。验证生成接口与本地 actor 训练不同；这些结果只支持所记录的协议与任务结果，不构成 vLLM 吞吐或参数更新结论。两组也不是检索器效果的直接对照实验。

6.3 模型执行中暴露的问题

除了最终分数，我检查了模型的工具调用和交付物反馈。以下采用两条实际轨迹说明执行问题，不把 judge 的个别判断扩展成模型的一般规律。完整定位见 submission/EXECUTION_ANALYSIS_EVIDENCE.md。

观察到的现象	实际证据	对技能与 harness 的启示
工具错误后重复执行，修复未先验证	贷款起草 new/sample-02 的生成脚本两次返回相同的 SyntaxError，随后轨迹仍完成	skill 可要求最小语法检查后再执行；harness 应区分工具故障与案件得分，并限制无变化重试
计算完成后过早标为“已核对”	地租起草 sample-01 被 judge 指出只做除法核对，没有反算舍入后的单价与总额是否一致	检查程序要包含正反算、容差和控制值选择；“做过计算”不足以证明一致
对材料冲突进行修复时改变了交付约束	同一地租轨迹把明确要求写成可选；贷款轨迹为满足比例限制而改变实际贷款金额，被相关 criterion 判失败	skill 应区分原始要求、发现的冲突和拟议修改；未经授权的改写要列为偏离，不可静默替代原条件
内容在成品和说明文件之间覆盖不一致	贷款 sample-02 的 memo 未明确识别基准替换条款需求；其他条款虽有讨论，仍未覆盖指定说明任务	建立“要求—来源—成品位置—未解决事项”的覆盖表，分别核对 memo、协议和附件
技能加载成功，整体任务结果仍退步	新增房地产起草技能在 8/8 条 new 轨迹进入 top-3，并替换原 Domain 技能；总体均值为负	技能要按整个检索 bundle 的净效果评价，不能按加载次数或文字合理性给 Writer 奖励

第一类是执行可恢复性问题。贷款样本共运行 69 个助手 turn，两次相同语法错误说明失败后缺少有效的局部校验；它不是最终 reward 本身。地租样本还出现过安装文档处理依赖时的网络解析失败，这属于环境能力，不能直接解释为模型法律推理差。若为此增加通用 skill，应要求先检查可用工具、优先使用现有依赖，并保留报错证据；harness 则负责记录工具状态和重试次数。当前尚未实现基于错误类型的自动恢复策略。

第二类是“生成出文件”与“完成要求”的差距。地租 sample-01 正常 finish，仍仅通过 60/67 个 criterion；贷款 new/sample-02 通过 78/84。文档可打开、脚本退出成功和模型的完成声明，只能证明执行链结束，不能代替逐项验收。判失败的某些条款涉及业务取舍，judge 也不等于最终法律裁决；这里能确认的是模型输出偏离了该任务的明确评价要求，需要把发现问题、建议谈判和实际改写分开呈现。

这些现象也说明初始 skill 不应只从成功轨迹总结“最佳步骤”。成功样本可提炼可靠程序，失败样本可提炼必要检查，但两者都只是候选证据。执行反馈经过抽象进入 seed 后，仍必须在其他案件上验证，才能知道它减少了遗漏，还是增加了冗余和干扰。下一步可按错误类型统计发生率，再对某一检查步骤单独做消融；本次两条示例还不足以证明具体错误由哪条技能导致。

7 反思：主要缺点、规模瓶颈与改进方向

首轮修订在两道题上都有提升，最后一版新增技能却带来轻微负收益。这两组结果让我更在意技能收益的解释：同一段步骤，在不同模型、不同库和不同检索组合下，可能有完全不同的作用。当前实验验证了生成、使用和评价能够串起来，对“怎样稳定积累有用经验”还只有很有限的认识。

7.1 归因仍然混有技能之间的替换效应

最后一版的新技能进入 top-3 时挤掉了原领域技能。测到的收益同时包含新增步骤的帮助、旧步骤退出的损失，以及更长上下文对求解的干扰。因此，Writer 的奖励能够评价这次库维护动作的净效果，却还不足以解释新技能具体好在哪里、坏在哪里。如果忽略这一点，很容易把检索竞争造成的退步误判成技能内容无用，或把旧技能被替换后的偶然改善归给新技能。

这也限制了当前的冲突管理。相同 ID 和共同修改目标容易检查，语义上的重复、条件冲突和互补关系则隐蔽得多。两条技能可能分别合理，放在一起却给出相反的处理顺序；也可能只有共同使用才有效。一次只发布一个动作便于归因，但随着库扩大，会越来越难依靠单条评价判断整个库的质量。

我更想先补一个小规模消融：在固定模型、案件和上下文预算下，分别观察新技能加入、旧技能退出、两者共同变化的结果，再回到自然检索下评价整库。这样可以保留“维护动作按净收益得分”的训练目标，同时给拒绝、修订或合并提供更具体的依据。技能的适用边界和相互关系，也应随这些证据更新。

7.2 规模瓶颈会先出现在评价，而不只是技能存储

当前每个候选只改一处，但要证明它有用，需要其他案件上的 old/new 求解。候选数、probe 题数、重复采样次数和执行模型数相乘，很快就会超过生成候选本身的成本。库越大，修改引起的检索替换也越多，局部改动未必还能用很小的验证集覆盖。持续增加候选数量，可能只是在增加未经充分验证的选择机会。

可以复用同一窗口、同一模型与父库条件下的旧库基线，并先排除结构无效、重复或明显泄漏的动作，再把评价预算用于有区别的候选。但筛选规则必须预先固定；若只给看起来有希望的候选挑简单 probe，会把评价成本降下来的同时把收益估高。对不确定的候选，应增加重复确认或保留旧版，不急于给出正负结论。

存储侧也需要逐步从“不断新增”转向维护有限的有效库。模型能力变化后，一些技能可能已经被内化，只剩下输入成本；另一些技能的价值来自少数特定场景。后续应记录使用条件、收益证据与最近验证的模型版本，支持合并、退役和按需加载。否则即使每次 ADD 看似合理，长期积累的库也可能越来越难用。

7.3 先提高评价的可信度，再扩大自动演化

当前只有少量同领域确认题，old/new 又是独立采样。微小正差可能来自随机波动，重复筛选还会放大“选中一个偶然赢家”的机会。即使 Writer 看不到 rubric，长期使用同一批 probe 和同一个 judge，也会通过发布反馈间接适应评价偏好。此时库在验证分数上持续进步，未必意味着新案件上的表现同步改善。

下一步我会先固定候选和规则，再用未参与选择的案件重复确认，并抽取一部分交付物做独立 judge 或人工盲评，核对源文件、业务约束与成品之间的对应。报告逐题波动和失败类型，比只报告一个平均增益更有解释力。首轮的正结果可以作为继续研究的线索，最后一版的负结果则提醒我，发布门槛应允许系统保持不变。

初始 seed 也应保留这种暂定态度。当前层级由人工预设，动作摘要又可能省略轨迹中段的重要修复；从来源评分提炼出的检查项还可能过度贴近已有任务。后续应检查哪些 seed 真正在未见任务上被使用、减少了哪类错误，再决定保留或重写。技能演化的价值最终要落在更可靠的任务执行上，库的条目数和修改次数都只能作为过程记录。

参考资料

[1] Voyager: An Open-Ended Embodied Agent with Large Language Models. https://arxiv.org/abs/2305.16291

[2] SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. https://arxiv.org/abs/2602.08234

[3] Evolving-RL: End-to-End Optimization of Experience-Driven Self-Evolving Capability within Agents. https://arxiv.org/abs/2605.10663

[4] VERL 源码基线：2ab35292；Harvey 源码基线：794557ac1。本文描述以交付工作树为准。